Machine Learning | Full-Stack | 2026

Airbnb Price Prediction Project

Interview Preparation Guide

> How to Explain Your ML Project with Confidence
>
> May 2026

Table of Contents

1. How to Introduce the Project 1

1.1 The 60-Second Elevator Pitch 1

1.2 The 3-Minute Structured Explanation 2

1.3 Key Talking Points to Emphasize 3

2. Project Architecture Overview 3

2.1 High-Level System Architecture 3

Data Layer 4

ML Pipeline Layer 4

API Layer 4

Web UI Layer 4

Deployment Layer 5

2.2 Technology Choices and Rationale 5

2.3 Technology Stack Summary 6

3. Data Pipeline Deep Dive 6

3.1 Data Profiling and Quality Assessment 6

3.2 Missing Value Strategy with Rationale 7

3.3 Feature Engineering Decisions 8

Amenity Parsing and Binary Flags 8

Date-to-Tenure Features 8

3.4 Encoding Strategy 9

4. Model Selection and Training 9

4.1 Model Comparison Approach 9

4.2 Why XGBoost Won 10

4.3 Hyperparameter Tuning Strategy 10

4.4 Evaluation Metrics and Interpretation 11

5. Production Deployment 11

5.1 FastAPI API Design Decisions 11

5.2 Web Frontend Technology Choices 12

5.3 Containerization and Deployment Strategy 12

6. Anticipated Interview Cross-Questions 12

6.1 Data and EDA Questions 13

6.2 Feature Engineering Questions 14

6.3 Model Selection Questions 15

6.4 Deployment and Production Questions 16

7. Key Concepts to Master 17

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. How to Introduce the Project

1.1 The 60-Second Elevator Pitch

Use this script when an interviewer asks, "Tell me about a project you've worked on." Keep it conversational and within 60 seconds.

"I built an end-to-end Airbnb price prediction system that uses machine learning to estimate listing prices based on 29 features like location, property type, and amenities. The dataset contained over 74,000 listings across major US cities. After thorough exploratory data analysis, I engineered features such as amenity counts and host tenure, applied one-hot and label encoding, and trained four models including XGBoost and LightGBM. XGBoost delivered the best performance with an R-squared of 0.785 after hyperparameter tuning with RandomizedSearchCV. I then wrapped the model in a FastAPI backend and built a Next.js frontend so users can input listing details and get real-time price predictions. The entire system is containerized with Docker for reproducible deployment."

1.2 The 3-Minute Structured Explanation

Use this structure for a detailed walkthrough. Each section should take roughly 30-40 seconds.

Problem Statement: "Pricing on Airbnb is challenging because hosts often lack a data-driven way to set competitive rates. I wanted to build a system that could predict optimal listing prices based on property characteristics and location data, helping both hosts and guests make informed decisions."

Data Overview: "The dataset contained 74,111 listings from cities including New York City, San Francisco, and Washington DC. It had 29 features covering property attributes, host information, geographic details, and amenity lists. The target variable was log-transformed price."

Approach: "I started with comprehensive EDA using distribution plots, correlation heatmaps, and city-wise analysis. For data cleaning, I dropped non-informative columns, parsed the host response rate from strings, converted date fields into tenure features, and handled missing values with median imputation for numerical features and 'Unknown' for categoricals. For feature engineering, I parsed the amenities JSON into individual binary flags and created an amenity count feature. I used one-hot encoding for low-cardinality categorical features and label encoding for high-cardinality ones like zip code, resulting in 83 total features."

Results: "I compared four models: Linear Regression as a baseline, Random Forest, XGBoost, and LightGBM. XGBoost achieved the best R-squared of 0.785 on the test set after tuning with RandomizedSearchCV over 50 iterations with 5-fold cross-validation. The 70/15/15 train-validation-test split ensured robust evaluation."

Deployment: "I built a FastAPI backend with /predict and /health endpoints, a Next.js and TypeScript frontend with Tailwind CSS, and containerized everything with Docker. The system can be deployed to cloud platforms like Heroku."

1.3 Key Talking Points to Emphasize

- **End-to-End Ownership:** You handled every stage from EDA to deployment, demonstrating full-stack ML engineering capability.
- **Data-Driven Decisions:** Every choice, from missing value imputation to encoding strategy, was justified by data characteristics, not arbitrary defaults.
- **Systematic Model Selection:** You compared four models with clear baselines and progressive complexity, showing structured thinking rather than trying a single model.
- **Production Readiness:** The project includes API design, web frontend, containerization, and testing, proving you think beyond notebooks.
- **Reproducibility:** YAML-based configuration, saved model artifacts with joblib, and feature name tracking demonstrate professional MLOps awareness.

2. Project Architecture Overview

2.1 High-Level System Architecture

The system follows a layered architecture pattern commonly used in production ML systems. Each layer has a clear responsibility and communicates through well-defined interfaces. The architecture flows from raw data through the ML pipeline, into a serving layer, and finally to user-facing applications.

Data Layer

The foundation of the system is the raw Airbnb listings dataset stored as CSV. The data layer includes the EDA notebook (eda.ipynb) for profiling and analysis, and the training notebook (training.ipynb) for iterative experimentation. Production data processing is handled by the Python module src/airbnb/train.py, which implements all cleaning, encoding, and feature engineering steps in a reproducible pipeline.

ML Pipeline Layer

The ML pipeline handles the complete lifecycle from raw data to trained model. Data cleaning drops non-informative columns and handles missing values. Feature engineering parses amenities JSON, creates binary flags, and converts date columns to tenure features. The encoding module applies one-hot encoding for five low-cardinality features and label encoding for two high-

cardinality features, producing 83 total features. The model training module trains four algorithms, performs hyperparameter tuning, and saves the best model along with metadata using joblib.

API Layer

The FastAPI backend (api/main.py) exposes three endpoints: /predict for real-time price predictions, /health for monitoring, and /features for retrieving the expected feature schema. The API uses Pydantic models for request validation, CORS middleware for frontend integration, and a build_feature_row function to transform incoming requests into the exact format the model expects.

Web UI Layer

The frontend is built with Next.js, TypeScript, and Tailwind CSS. The PredictionForm component collects listing details from users, and the ResultCard component displays predictions with confidence indicators. TypeScript types ensure type safety between the frontend and API contract.

Deployment Layer

Containerization uses Docker with a multi-stage Dockerfile and docker-compose.yml for orchestration. Deployment targets include cloud platforms via Heroku yaml configuration and a Procfile for process management.

2.2 Technology Choices and Rationale

Every technology was selected for a specific reason aligned with production requirements and industry best practices.

- **Python + scikit-learn / XGBoost / LightGBM:** These are the industry-standard libraries for tabular ML. scikit-learn provides robust preprocessing and baseline models, while XGBoost and LightGBM offer state-of-the-art gradient boosting performance.
- **FastAPI:** Chosen for its automatic OpenAPI documentation, async support, and Pydantic integration. It is significantly faster than Flask for API serving and provides built-in request validation.
- **Next.js + TypeScript:** Next.js provides server-side rendering, routing, and a component-based architecture. TypeScript ensures type safety and reduces runtime errors, which is critical when interfacing with an API.
- **Tailwind CSS:** Utility-first CSS enables rapid UI development without writing custom stylesheets, and produces a clean, professional interface with consistent design tokens.
- **Docker:** Containerization ensures the environment is identical across development, testing, and production, eliminating "works on my machine" issues.

- **YAML Configuration:** Centralized config management allows easy parameter tuning without modifying code, supporting experimentation and deployment flexibility.

2.3 Technology Stack Summary

Table 1: Complete Technology Stack

Layer	Technology	Purpose
Data / EDA	Jupyter Notebooks, pandas, matplotlib, seaborn	Exploratory analysis and profiling
Data Processing	Python, scikit-learn	Cleaning, encoding, feature engineering
ML Models	LinearRegression, RandomForest, XGBoost, LightGBM	Model training and comparison
Hyperparameter Tuning	RandomizedSearchCV (scikit-learn)	Optimal parameter search with 5-fold CV
Model Persistence	joblib, JSON	Artifact saving and feature metadata
API Backend	FastAPI, Pydantic, uvicorn	RESTful prediction service
Web Frontend	Next.js, TypeScript, Tailwind CSS	User interface for predictions
Containerization	Docker, docker-compose	Reproducible deployment
Deployment	Heroku, Procfile	Cloud hosting and process management
Testing	pytest	Unit tests for pipeline and training
Configuration	YAML (config.yaml)	Centralized parameter management

3. Data Pipeline Deep Dive

3.1 Data Profiling and Quality Assessment

The first step was comprehensive data profiling using the EDA notebook. This involved examining data types, missing value patterns, distribution characteristics, and inter-feature correlations. The missingno library was used to visualize missing data patterns through matrix plots and bar charts, revealing which columns had significant gaps and whether missingness was random or structured.

Distribution plots were created for numerical features to identify skewness, which informed the decision to use log-transformed price as the target variable. Correlation heatmaps revealed multicollinearity patterns and highlighted features with the strongest relationships to price. City-wise analysis showed significant price variation across markets, confirming that geographic features would be important predictors.

Outlier detection was performed using IQR-based methods and visualization through box plots. Rather than automatically removing outliers, the decision was made to retain them as they likely represent genuine luxury listings or unique properties that the model should learn to predict.

3.2 Missing Value Strategy with Rationale

The missing value strategy was tiered and driven by the nature of each feature:

- **Column Dropping (4 columns):** The columns 'id', 'thumbnail_url', 'name', and 'description' were dropped because they are identifiers or free-text fields that do not contribute meaningful predictive signal for a tabular ML model without NLP processing.
- **String Parsing:** Host response rate was stored as strings like '100%' and was parsed into numeric values to be treated as a continuous feature.
- **Boolean Mapping:** Boolean columns were mapped to 0/1 integer values, which is the standard representation for tree-based models.
- **Date Conversion:** Date columns like 'host_since' were converted to tenure (days since listing) and recency features, which are more meaningful numeric representations than raw timestamps.
- **Numerical Imputation (Median):** Median was chosen over mean for numerical missing values because it is robust to outliers. In housing data, extreme values are common, and the median provides a more stable central tendency estimate.
- **Categorical Imputation ('Unknown'):** For categorical features, 'Unknown' was used instead of mode imputation because it explicitly signals the absence of information rather than introducing potential bias through the most common category.
- **Row Dropping (74K to 42K):** Rows with remaining null values after imputation were dropped. This aggressive cleaning step was necessary to maintain model quality, and the resulting 42K rows still provided sufficient statistical power for training.

3.3 Feature Engineering Decisions

Amenity Parsing and Binary Flags

The amenities column contained JSON-style strings listing available amenities for each listing. Using Python's ast.literal_eval, these were parsed into Python lists. An amenity_count feature was created to capture the total number of amenities, reflecting the overall quality tier of a listing.

Additionally, 14 binary flags were created for high-value amenities that showed meaningful correlation with price, such as WiFi, air conditioning, kitchen, washer/dryer, free parking, pool, gym, and others.

The rationale for binary flags over one-hot encoding all amenities was dimensionality management. The raw amenity lists contained hundreds of unique values, and one-hot encoding all of them would create excessive sparse features. Selecting the 14 most predictive amenities kept the feature space manageable while capturing the most important amenity signals.

Date-to-Tenure Features

Date columns like 'host_since' and 'first_review' were converted into tenure features by calculating the number of days between the date and a reference point. This transformation captures the experience level of hosts and the maturity of listings as continuous numeric features, which are more useful for regression models than raw date strings.

3.4 Encoding Strategy

The encoding strategy was chosen based on cardinality analysis of each categorical feature:

- **One-Hot Encoding (5 features):** Applied to city, room_type, property_type, bed_type, and cancellation_policy. These features have relatively low cardinality, so one-hot encoding is efficient and avoids imposing ordinal relationships that do not exist.
- **Label Encoding (2 features):** Applied to zipcode and neighbourhood. These features have very high cardinality (hundreds or thousands of unique values). One-hot encoding would create massive sparse matrices, so label encoding was chosen as a compact representation. Tree-based models like XGBoost handle label-encoded features well because they can learn non-linear decision boundaries.

Table 2: Feature Engineering Summary

Transformation	Input	Output	Technique
Column removal	29 features	25 features	Domain judgment
String parsing	host_response_rate (string)	Numeric rate	Percentage parsing
Boolean mapping	True/False columns	0/1 integers	Direct mapping
Date conversion	host_since, dates	Tenure in days	Date arithmetic
Numerical imputation	Null numeric values	Median-filled	Robust central tendency
Categorical imputation	Null categorical	"Unknown" category	Explicit missing indicator

Transformation	Input	Output	Technique
Amenity parsing	JSON strings	14 binary flags + count	ast.literal_eval + feature selection
One-hot encoding	5 low-cardinality features	Expanded columns	pandas get_dummies
Label encoding	2 high-cardinality features	Integer labels	sklearn LabelEncoder
Final feature count	25 base features	83 total features	Combined transformations

4. Model Selection and Training

4.1 Model Comparison Approach

The modeling strategy followed a progressive complexity approach, starting with a simple baseline and advancing to more sophisticated ensemble methods. This methodology ensures that each additional model's complexity is justified by measurable performance improvement.

Table 3: Model Comparison Results

Model	Type	R-squared (Test)	Key Strength
Linear Regression	Linear / Baseline	0.677	Interpretability, fast training
Random Forest	Bagging Ensemble	0.765	Handles non-linearity, robust to outliers
XGBoost	Gradient Boosting	0.785	Best accuracy, regularization built-in
LightGBM	Gradient Boosting	0.782	Fast training, handles large datasets

The train/validation/test split used a 70/15/15 ratio. This was chosen over a simpler 80/20 split because having a separate validation set allowed for honest hyperparameter tuning without contaminating the test set, which was reserved exclusively for final performance evaluation.

4.2 Why XGBoost Won

XGBoost outperformed all other models for several reasons. First, gradient boosting builds trees sequentially, with each tree correcting the errors of the previous ensemble, leading to superior predictive accuracy. Second, XGBoost includes L1 and L2 regularization by default, which prevents overfitting, a critical advantage over Random Forest when the feature space is relatively wide (83

features). Third, XGBoost handles missing values natively, though in this case explicit imputation was already performed. Fourth, the algorithm's built-in parallelization and efficient memory usage make it practical for production deployment.

LightGBM performed very closely (R-squared of 0.782), which is expected since both are gradient boosting frameworks. LightGBM excels with extremely large datasets through its leaf-wise tree growth, but for this dataset size of 42K rows, XGBoost's depth-wise growth provided slightly better generalization.

4.3 Hyperparameter Tuning Strategy

Hyperparameter tuning used RandomizedSearchCV with 50 iterations and 5-fold cross-validation. Randomized search was chosen over grid search because it explores a wider hyperparameter space more efficiently. With 50 iterations, it provides good coverage of the search space without the exponential computational cost of exhaustive grid search.

Key hyperparameters tuned included the number of estimators (trees), maximum depth, learning rate, subsample ratio, column subsample ratio, and regularization parameters (alpha for L1, lambda for L2). The 5-fold cross-validation ensures that the reported performance is reliable and not dependent on a particular train-validation split.

4.4 Evaluation Metrics and Interpretation

The primary evaluation metric was R-squared (coefficient of determination), which measures the proportion of variance in the target variable explained by the model. An R-squared of 0.785 means the model explains approximately 78.5% of the variance in log-transformed Airbnb prices, which is a strong result for a tabular regression task.

R-squared was chosen over RMSE or MAE because it provides a scale-independent measure of fit quality, making it easier to communicate model performance. However, RMSE was also tracked during development to understand prediction error in the original price units (after exponentiating the log-transformed predictions). The train/validation/test split methodology ensured that the test set R-squared reflects genuine generalization performance, not memorization of training data.

5. Production Deployment

5.1 FastAPI API Design Decisions

The API layer was designed with production best practices in mind. The /predict endpoint accepts listing features as JSON, validates them using Pydantic models, transforms them into the model's expected format using build_feature_row, and returns a price prediction. The /health endpoint provides a simple liveness check for monitoring and load balancer health checks. The /features

endpoint returns the expected feature schema, enabling frontend clients to dynamically generate input forms.

Pydantic models ensure type safety and automatic validation of incoming requests, rejecting malformed data before it reaches the model. CORS middleware was configured to allow cross-origin requests from the Next.js frontend during development and production. The separation between request format and model input format through build_feature_row decouples the API contract from the internal model representation, allowing either to evolve independently.

5.2 Web Frontend Technology Choices

Next.js was chosen for its server-side rendering capability, automatic code splitting, and file-based routing. TypeScript was used throughout to ensure type safety and reduce bugs. The PredictionForm component provides a structured input interface, and the ResultCard component displays predictions with clear formatting. Tailwind CSS enabled rapid styling without custom CSS files.

The frontend follows a component-based architecture where TypeScript types mirror the API's Pydantic models, creating an end-to-end type-safe contract between the frontend and backend. This approach catches data shape mismatches at compile time rather than at runtime.

5.3 Containerization and Deployment Strategy

The Dockerfile uses a multi-stage build pattern: the first stage installs dependencies, and the second stage copies only the necessary artifacts, resulting in a smaller production image. The docker-compose.yml defines services for both the API and frontend, enabling local development that mirrors production.

For cloud deployment, a Heroku yaml configuration and Procfile specify the process types and build configuration. This allows one-command deployment to Heroku's platform. The containerized approach ensures that the model environment, including library versions and system dependencies, is identical across all deployment targets, eliminating a major class of production bugs.

Table 4: Deployment Architecture Summary

Component	Technology	Configuration
API Server	FastAPI + uvicorn	Auto-scaling, async I/O
Web Server	Next.js (built-in)	Static export or SSR mode
Container Build	Docker multi-stage	Minimized production image
Orchestration	docker-compose	Local multi-service development
Cloud Platform	Heroku	Procfile + yaml configuration

Component	Technology	Configuration
Health Monitoring	/health endpoint	Liveness check for load balancers

6. Anticipated Interview Cross-Questions

6.1 Data and EDA Questions

Q: Why did you choose log_price as the target variable instead of raw price?

A: Price data in real estate and hospitality is typically right-skewed with a long tail of expensive listings. Taking the log transform normalizes the distribution, reduces the influence of extreme outliers, and makes the relationship between features and price more linear. This improves model performance because linear and tree-based models generally perform better when the target has a roughly normal distribution. After prediction, the result can be exponentiated to return to the original price scale.

Q: What was the biggest data quality issue you encountered and how did you handle it?

A: The most significant issue was the volume of missing values across multiple columns. Rather than applying a single imputation strategy uniformly, I analyzed each feature's missingness pattern and data type individually. Numerical features received median imputation because it is robust to outliers, categorical features received an 'Unknown' category to explicitly signal missingness, and dates were converted to tenure features before imputation. Rows with remaining nulls after this targeted approach were dropped, reducing the dataset from 74K to 42K rows while preserving data quality.

Q: How did you decide which outliers to keep or remove?

A: I chose to retain outliers rather than remove them. In the context of Airbnb listings, extremely high prices often represent genuine luxury properties or premium locations that are valid data points the model should learn from. Removing them would artificially limit the model's prediction range and reduce its usefulness for high-end listings. I verified this decision by checking that the model's residuals were roughly homogeneous across the price range, indicating that outliers were not disproportionately degrading model quality.

Q: What would you do differently if you had more time for EDA?

A: I would perform geospatial analysis using latitude and longitude coordinates to create neighborhood-level features and distance-to-points-of-interest features. I would also conduct temporal analysis to identify seasonal pricing patterns. Additionally, I would use SHAP values to understand feature interactions and guide further feature engineering. Text analysis of

listing descriptions could also extract sentiment and keyword features that correlate with pricing.

6.2 Feature Engineering Questions

Q: Why did you select exactly 14 amenities for binary flags instead of using all of them?

A: I analyzed the correlation between individual amenities and price, selecting those that showed meaningful predictive power. Using all amenities would have created hundreds of sparse binary features, which increases dimensionality without proportional information gain. High-dimensional sparse features can lead to overfitting, increased training time, and reduced model interpretability. The 14 selected amenities, including WiFi, air conditioning, kitchen, and pool, captured the most important amenity-related price signals.

Q: Why one-hot encoding for some features and label encoding for others?

A: The decision was based on cardinality. Low-cardinality features like city (6 values) and room_type (3 values) are well-suited for one-hot encoding because they create only a few additional columns without significant dimensionality increase. One-hot encoding avoids imposing a false ordinal relationship. High-cardinality features like zipcode (hundreds of values) would create an impractical number of columns with one-hot encoding, so label encoding was used. Tree-based models handle label-encoded features effectively because they can partition on integer values without assuming ordinality.

Q: How did you handle the amenities JSON parsing and why ast.literal_eval?

A: The amenities column contained string representations of Python lists, not actual JSON. I used ast.literal_eval because it safely evaluates Python literal expressions including lists, tuples, and dictionaries. Unlike eval(), it does not execute arbitrary code, making it secure. After parsing, I created binary flags by checking whether each of the 14 selected amenities appeared in the parsed list.

Q: What other features would you engineer if you had more data?

A: I would create interaction features between location and property type, text features from listing descriptions using TF-IDF or embeddings, geospatial features like distance to city center or transit, temporal features like day-of-week and seasonality indicators, and host reputation features aggregating review scores. I would also explore clustering neighborhoods into price tiers as a derived categorical feature.

6.3 Model Selection Questions

Q: Why not use a neural network for this task?

A: For tabular data with 83 features and 42K rows, tree-based ensemble methods like XGBoost generally outperform neural networks. Neural networks require significantly more data to

generalize well, are harder to tune, and lack the built-in interpretability of tree-based models. Research has consistently shown that gradient boosting is the state-of-the-art approach for structured tabular data, while neural networks excel with unstructured data like images, text, and audio.

Q: How do you prevent overfitting in your XGBoost model?

A: Several mechanisms work together. L1 and L2 regularization penalize complex trees. Subsample and colsample_bytree introduce randomness that prevents the model from memorizing training data. Early stopping based on validation set performance halts training when additional trees no longer improve generalization. The train/validation/test split ensures that hyperparameter tuning does not leak test information. Finally, the moderate tree depth prevents the model from creating overly specific leaf nodes.

Q: Why RandomizedSearchCV instead of GridSearchCV or Bayesian optimization?

A: RandomizedSearchCV offers the best balance between exploration and computational cost for this project. GridSearchCV is computationally prohibitive when tuning multiple hyperparameters with large ranges because it evaluates every combination. Bayesian optimization is more sample-efficient but adds implementation complexity. With 50 iterations, RandomizedSearchCV provides sufficient exploration of the hyperparameter space at a fraction of the computational cost of grid search, and it scales gracefully as more hyperparameters are added.

6.4 Deployment and Production Questions

Q: How would you handle model drift in production?

A: I would implement a monitoring system that tracks prediction distribution and feature distributions over time. If the input feature distribution shifts significantly from the training data, an alert would be triggered. I would establish a retraining pipeline that runs periodically or when drift is detected, using the same YAML-configured pipeline. A/B testing between the current and retrained models would validate that the new model improves performance before full deployment.

Q: How would you scale this system for millions of predictions per day?

A: I would deploy the model behind a load balancer with multiple API instances. For higher throughput, I could batch predictions and use a message queue system. If latency requirements are very strict, I would convert the model to ONNX format for optimized inference or use a serving framework like TensorFlow Serving or Triton Inference Server. Caching predictions for similar inputs and pre-computing features for known listings would further reduce computational load.

Q: What are the security considerations for this API?

A: The API needs input validation through Pydantic to prevent injection attacks. Rate limiting prevents abuse and denial-of-service attacks. CORS should be configured restrictively to allow only known frontend origins. Authentication and authorization would be needed if the API serves premium features. HTTPS is essential for encrypting data in transit. The model file and configuration should not be publicly accessible.

7. Key Concepts to Master

The following technical concepts are essential for defending this project in an interview. Each concept directly relates to a design decision or implementation detail in the Airbnb price prediction system.

Table 5: Critical Technical Concepts

Concept	Why It Matters for This Project
Log Transformation	Explains why log_price was used as the target variable. Understand when and why log transforms are applied (skewed distributions, multiplicative relationships, homoscedasticity).
Missing Value Imputation Strategies	Central to the data cleaning pipeline. Be able to compare median vs. mean vs. mode vs. KNN imputation, and explain the trade-offs of each approach.
One-Hot vs. Label Encoding	Directly relevant to the encoding decisions. Know when each is appropriate based on cardinality, model type, and the risk of imposing false ordinal relationships.
Train/Validation/Test Splitting	Justifies the 70/15/15 split. Understand why separate validation and test sets are needed, and the risk of data leakage through improper splitting.
Bias-Variance Tradeoff	Explains why XGBoost outperformed simpler models while regularization prevented overfitting. Know how to diagnose underfitting vs. overfitting.
Cross-Validation	Central to hyperparameter tuning with 5-fold CV. Understand k-fold, stratified, and the statistical benefits of CV over a single split.
Gradient Boosting (XGBoost / LightGBM)	The core algorithm behind the best-performing model. Know how sequential tree building, learning rate, and regularization contribute to performance.

Concept	Why It Matters for This Project
Randomized Search vs. Grid Search	Justifies the hyperparameter tuning approach. Understand the computational trade-offs and why randomized search is preferred for high-dimensional search spaces.
R-squared and Regression Metrics	The primary evaluation metric. Be able to interpret R-squared, compare it to RMSE and MAE, and explain what the 0.785 result means in practical terms.
API Design with Pydantic	Relevant to the FastAPI backend. Understand request validation, type safety, and how Pydantic models enforce data contracts between frontend and backend.
Docker and Containerization	Essential for the deployment strategy. Know why containerization ensures reproducibility, how multi-stage builds reduce image size, and the role of docker-compose in orchestration.
Model Monitoring and Drift Detection	A forward-looking production consideration. Understand feature drift, concept drift, and strategies for retraining models in production environments.

Preparation Tip: For each concept above, practice explaining it in 30 seconds to a peer. The ability to concisely articulate technical decisions is often more impressive than the decisions themselves.

Good luck with your interviews! This preparation guide covers the technical depth and communication strategy needed to present this project with confidence and professionalism.